

**LAPORAN PRAKTIKUM BOX MODEL, SELECTOR& SPECIFITY,WARNA,
TIPOGRAFI, UNIT, FLEXBOX**



Disusun Oleh:

Nama : Naula Rizwana
Nim : 2025573010058
Kelas : TI-1C
Dosen : Muhammad Reza Zulman, S.ST., M.Sc

**JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
PRODI TEKNIK INFORMATIKA
POLITEKNIK NEGERI LHOKSEUMAWE
2025-2026**

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Struktur Data dan Algoritma Modul 3 yang berjudul "*Function dan Array dalam JavaScript*" dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan. Dalam praktikum ini, penulis mempelajari berbagai konsep dasar dalam pemrograman JavaScript, khususnya mengenai function, arrow function, callback, array, method iterasi array, serta rekursi. Materi-materi tersebut merupakan dasar penting dalam pengembangan program yang efisien, terstruktur, dan mudah dikembangkan di masa mendatang.

Selama proses praktikum, penulis tidak hanya memahami teori, tetapi juga langsung mengimplementasikannya melalui berbagai latihan dan tugas yang diberikan. Hal ini membantu penulis dalam meningkatkan kemampuan logika pemrograman serta pemecahan masalah secara sistematis.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca serta menjadi referensi dalam memahami konsep Function dan Array dalam JavaScript.

Naula Rizwana.

TUJUAN PRAKTIKUM

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar function dalam bahasa pemrograman JavaScript
2. Mampu membuat dan menggunakan function dengan parameter dan nilai kembalian
3. Memahami penggunaan arrow function sebagai sintaks modern dalam JavaScript
4. Memahami konsep callback dan penerapannya dalam program
5. Mampu menggunakan array sebagai struktur data untuk menyimpan dan mengelola data
6. Memahami serta mengimplementasikan method array seperti *forEach*, *map*, *filter*, dan *reduce*

Manfaat Praktikum

Adapun manfaat dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Menambah pemahaman mahasiswa tentang konsep dasar pemrograman JavaScript
2. Melatih kemampuan dalam membuat dan mengelola function
3. Meningkatkan kemampuan dalam mengolah data menggunakan array
4. Membantu memahami penggunaan method array secara efektif
5. Melatih logika pemrograman melalui penerapan rekursi
6. Menjadi dasar dalam pembuatan program yang lebih kompleks

Buat folder praktikum-3/ dan masuk ke dalamnya.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)  
○ $ mkdir praktikum-3
```

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)  
● $ cd praktikum-3
```

Inisialisasi npm project.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)  
○ $ npm init-y
```

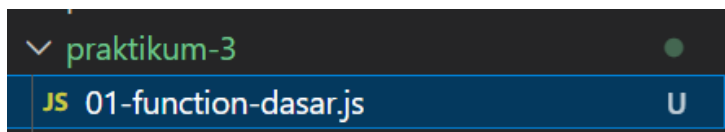
Bagian 1 — Function

Function adalah blok kode yang diberi nama dan dapat dipanggil (dieksekusi) kapan saja dan berulang kali. Function (fungsi) adalah sekumpulan kode yang dirancang untuk melakukan suatu tugas tertentu dan dapat dipanggil kembali kapan saja dibutuhkan. Function merupakan salah satu konsep dasar dalam pemrograman yang bertujuan untuk membuat kode lebih terstruktur, efisien, dan mudah dipahami.

Dalam JavaScript, function digunakan untuk mengelompokkan perintah-perintah tertentu sehingga tidak perlu menulis kode yang sama secara berulang. Hal ini sesuai dengan prinsip **DRY (Don't Repeat Yourself)** dalam pemrograman.

Langkah 1 Pastikan kamu berada di dalam folder praktikum-3/. Buat file baru bernama 01-function-dasar.js.

```
touch 01-function-dasar.js
```



Langkah 2 Buka file 01-function-dasar.js di VS Code, lalu ketikkan kode berikut.

```

praktikum-3 > JS 01-function-dasar.js > ...
1  function sapa() {
2      console.log("Halo! Selamat datang di praktikum JavaScript.");
3  }
4  sapa();
5  sapa();
6
7  function sapaNama(nama) {
8      console.log(`Halo, ${nama}! Selamat belajar.`);
9  }
10 sapaNama("Budi");
11 sapaNama("Siti");
12
13 function tambah(angka1, angka2) {
14     const hasil = angka1 + angka2;
15     return hasil;
16 }
17 const hasilPenjumlahan = tambah(10, 25);
18 console.log("10 + 25 =", hasilPenjumlahan);
19 console.log("7 + 13 =", tambah(7, 13));
20
21 function hitung(nilai, pengali = 2) {
22     return nilai * pengali;
23 }
24 console.log(hitung(5));

```

Langkah 3 Simpan file, lalu jalankan dari terminal.

```

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main) ...
● $ node 01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Budi! Selamat belajar.
Halo, Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja

```

Latihan 1

```
38 // Function penjumlahan (misal sudah ada)
39 function tambah(a, b) {
40     return a + b;
41 }
42
43 // 2. Function kurang
44 function kurang(a, b) {
45     return a - b;
46 }
47
48 // 3. Function kali
49 function kali(a, b) {
50     return a * b;
51 }
52
53 // 4. Function bagi
54 function bagi(a, b) {
55     if (b === 0) {
56         console.log("Error: tidak bisa membagi dengan nol");
57         return null;
58     }
59     return a / b;
60 }
61
62 // 5. Function kalkulator
63 function kalkulator(a, operator, b) {
64     switch (operator) {
65         case "+":
66             return tambah(a, b);
```

15

5

50

2

Error: tidak bisa membagi dengan nol

null

Bagian 2 — Arrow Function & Callback

Arrow function adalah sintaks penulisan function yang lebih ringkas, diperkenalkan di ES6 (2015). Alih-alih menulis function nama(params) { ... }, kita bisa menulis const nama = (params) => { ... }.

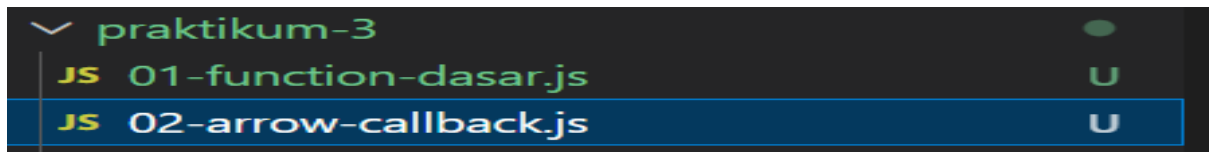
Aturan penulisan arrow function:

- Jika hanya ada satu parameter, kurung bisa dihilangkan: $x \Rightarrow x * 2$
- Jika body hanya satu baris dan langsung return, kurung kurawal dan return bisa dihilangkan: (a, b) $\Rightarrow a + b$
- Jika body lebih dari satu baris, tetap gunakan kurung kurawal dan return.

Callback adalah sebuah function yang dikirimkan sebagai argumen ke function lain. Function penerima akan memanggil callback tersebut di waktu yang tepat. Ini adalah konsep yang sangat fundamental di JavaScript — hampir semua event handling, timer, dan method array menggunakan callback.

Langkah 1 Buat file baru bernama 02-arrow-callback.js di dalam folder praktikum-3/.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch 02-arrow-callback.js
```



Langkah 2 Buka file 02-arrow-callback.js di VS Code, lalu ketikkan kode berikut.

```
praktikum-3 > JS 02-arrow-callback.js > ...
1  function kuadratBiasa(x) {
2      return x * x;
3  }
4  const kuadrat = (x) => {
5      return x * x;
6  };
7  const kuadratRingkas = (x) => x * x;
8
9  console.log("=== Perbandingan Penulisan ===");
10 console.log("Biasa      :", kuadratBiasa(5));
11 console.log("Arrow     :", kuadrat(5));
12 console.log("Ringkas   :", kuadratRingkas(5));
13
14 const luas = (panjang, lebar) => panjang * lebar;
15 const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;
16
17 console.log("Luas 4x6 :", luas(4, 6));
18 console.log(salam("Budi", "Pagi"));
19
20 function lakukanOperasi(angka, operasiCallback) {
21     console.log(`Angka awal: ${angka}`);
22     const hasil = operasiCallback(angka);
```

Langkah 3 Simpan file, lalu jalankan dari terminal.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
● $ node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa      : 25
Arrow      : 25
Ringkas    : 25
Luas 4x6   : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
Ringkas : 25
```

Latihan 2

```
42 // 2. Arrow function keHurufBesar
43 const keHurufBesar = (teks) => teks.toUpperCase();
44
45 // 3. Arrow function tambahSeru
46 const tambahSeru = (teks) => teks + "!!!";
47
48 // 4. Arrow function hitungKata
49 const hitungKata = (teks) => teks.split(" ").length;
50
51 // 5. Function prosesKalimat
52 function prosesKalimat(kalimat, transformasiCallback) {
53   const hasil = transformasiCallback(kalimat);
54   console.log(hasil);
55 }
56
57 // 6. Uji function
58 const kalimat = "belajar javascript itu menyenangkan";
59
60 prosesKalimat(kalimat, keHurufBesar);
61 // Output: BELAJAR JAVASCRIPT ITU MENYENANGKAN
62
63 prosesKalimat(kalimat, tambahSeru);
64 // Output: belajar javascript itu menyenangkan!!!
65
```



```
=== setTimeout (Callback) ===  
Pesan 1: Sebelum timer  
Pesan 2: Setelah mendaftarkan timer  
Pesan 1: Sebelum timer  
Pesan 2: Setelah mendaftarkan timer  
Pesan 2: Setelah mendaftarkan timer  
BELAJAR JAVASCRIPT ITU MENYENANGKAN  
belajar javascript itu menyenangkan!!!  
4  
belajar javascript itu menyenangkan!!!  
4  
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

Bagian 3 — Array Dasar

Array adalah struktur data yang menyimpan kumpulan nilai dalam satu variabel, berurutan dan terindeks. Bayangkan array seperti sebuah rak buku bernomor — setiap buku (elemen) memiliki posisi (indeks) yang dimulai dari angka 0.

Array di JavaScript bersifat dinamis — ukurannya bisa berubah sewaktu-waktu. Array juga bisa menyimpan tipe data yang berbeda-beda dalam satu array (meskipun praktik terbaik adalah menyimpan satu jenis data per array).

Langkah 1 Buat file baru bernama 03-array-dasar.js di dalam folder praktikum-3/.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)  
● $ touch 03-array-dasar.js.
```

▼ praktikum-3	●
JS 01-function-dasar.js	U
JS 02-arrow-callback.js	U
JS 03-array-dasar.js	U

Langkah 2 Buka file 03-array-dasar.js di VS Code, lalu ketikkan kode berikut.

```
praktikum-3 > JS 03-array-dasar.js > ...
1  const mahasiswa = ["Budi", "Siti", "Ahmad", "Rina"];
2  const nilai = [85, 92, 78, 95, 88];
3  console.log("=== Array Awal ===");
4  console.log("Mahasiswa:", mahasiswa);
5  console.log("Nilai      :", nilai);
6  console.log("Jumlah mahasiswa:", mahasiswa.length);
7
8  console.log("\n=== Akses Elemen ===");
9  console.log("Elemen pertama :", mahasiswa[0]);
10 console.log("Elemen ketiga  :", mahasiswa[2]);
11 console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);
12
13 mahasiswa[1] = "Siti Rahayu";
14 console.log("\nSetelah diubah:", mahasiswa);
15
16 console.log("\n=== Manipulasi Array ===");
17 mahasiswa.push("Doni");
18 console.log("Setelah push    :", mahasiswa);
19
20 mahasiswa.unshift("Zahra");
21 console.log("Setelah unshift  :", mahasiswa);
22
23 const dihapusAkhir = mahasiswa.pop();
24 console.log("Dihapus (pop)   :", dihapusAkhir);
25 console.log("Setelah pop     :", mahasiswa);
```

Langkah 3 Simpan file, lalu jalankan dari terminal.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3
● $ node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai      : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga  : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push    : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop)   : Doni
Setelah pop     : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift   : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
```

Latihan 3

```
40 // 2. Membuat array daftarBelanja
41 let daftarBelanja = ["Beras", "Gula", "Minyak", "Telur", "Kopi"];
42
43 // 3. Menampilkan isi daftar dengan nomor urut
44 for (let i = 0; i < daftarBelanja.length; i++) {
45   console.log(`${i + 1}. ${daftarBelanja[i]}`);
46 }
47
48 // 4. Menambahkan 2 item baru
49 daftarBelanja.push("Susu");
50 daftarBelanja.push("Roti");
51
52 // 5. Menghapus item pertama
53 let itemDihapus = daftarBelanja.shift();
54 console.log("Item yang dihapus:", itemDihapus);
55
56 // 6. Mengecek apakah 'Susu' ada
57 let cekSusu = daftarBelanja.includes("Susu");
58 console.log("Apakah ada 'Susu' di daftar?", cekSusu);
59
60 // 7. Menampilkan jumlah item akhir
61 console.log("Jumlah item akhir:", daftarBelanja.length);
```

=== Pencarian ===

Indeks Ahmad : 2

Ada Rina? : true

Ada Budi? : true

Nilai asli : [85, 92, 78, 95, 88]

Slice [1,4] : [92, 78, 95]

1. Beras

2. Gula

3. Minyak

4. Telur

5. Kopi

Item yang dihapus: Beras

Apakah ada 'Susu' di daftar? true

Jumlah item akhir: 6

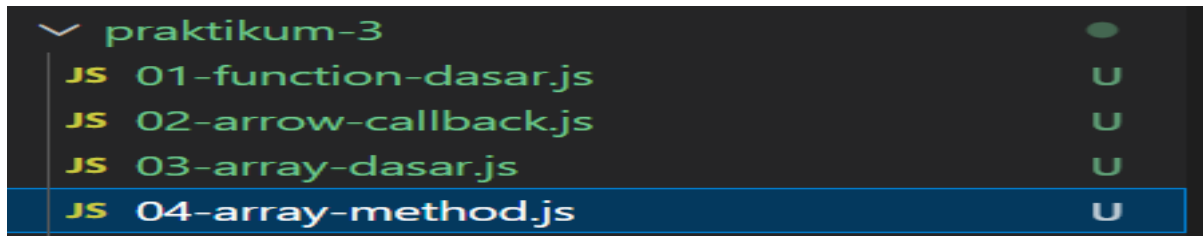
Bagian 4 — Method Iterasi Array

Method iterasi array adalah sekumpulan fungsi bawaan dalam JavaScript yang digunakan untuk melakukan perulangan (iterasi) pada setiap elemen array secara otomatis. Method ini memungkinkan programmer untuk memproses data dalam array dengan cara yang lebih ringkas, efisien, dan mudah dibaca dibandingkan menggunakan perulangan biasa seperti for atau while. Pada method iterasi, biasanya digunakan callback function, yaitu function yang dijalankan untuk setiap elemen dalam array

- `forEach(callback)` — Menjalankan callback untuk setiap elemen. Tidak mengembalikan nilai apapun. Digunakan hanya untuk efek samping (side effects) seperti mencetak atau menyimpan ke database.
- `map(callback)` — Membuat array baru dengan menerapkan callback ke setiap elemen. Array asli tidak berubah. Digunakan untuk mentransformasi setiap elemen.
- `filter(callback)` — Membuat array baru yang hanya berisi elemen yang memenuhi kondisi (callback mengembalikan true). Array asli tidak berubah. Digunakan untuk menyaring elemen.
- `reduce(callback, nilaiAwal)` — Mereduksi seluruh array menjadi satu nilai tunggal. Callback menerima dua argumen: accumulator (nilai yang terakumulasi) dan currentValue (elemen saat ini).

Langkah 1 Buat file baru bernama 04-array-method.js di dalam folder praktikum-3/.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
$ touch 04-array-method.js
```



Langkah 2 Buka file 04-array-method.js di VS Code, lalu ketikkan kode berikut.

```
praktikum-3 > JS 04-array-method.js > produk
1  const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];
2
3  console.log("=== forEach: Tampilkan Semua Nilai ===");
4  nilaiMahasiswa.forEach((nilai, indeks) => {
5    console.log(`Mahasiswa-${indeks + 1}: ${nilai}`);
6  });
7
8  const gradeHuruf = nilaiMahasiswa.map((nilai) => {
9    if (nilai >= 90) return "A";
10   if (nilai >= 80) return "B";
11   if (nilai >= 70) return "C";
12   if (nilai >= 60) return "D";
13   return "E";
14 });
15
16 console.log("\n=== map: Nilai ke Grade ===");
17 console.log("Nilai :", nilaiMahasiswa);
18 console.log("Grade :", gradeHuruf);
19
20 const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
21 const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);
22
23 console.log("\n=== filter: Lulus dan Tidak Lulus ===");
24 console.log("Semua nilai :", nilaiMahasiswa);
25 console.log("Nilai lulus :", nilaiLulus);
26 console.log("Nilai gagal :", nilaiGagal);
```

Langkah 3 Simpan file, lalu jalankan dari terminal.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
● $ node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
Mahasiswa-1: 75
Mahasiswa-2: 90
Mahasiswa-3: 55
Mahasiswa-4: 82
Mahasiswa-5: 68
Mahasiswa-6: 95
Mahasiswa-7: 48
Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]
```

Latihan 4

```
praktikum-3 > JS 04-array-method.js > [🔗] produk
47 // 2-3. Deklarasi array of objects
48 const produk = [
49   { nama: "Laptop", harga: 8500000, stok: 5 },
50   { nama: "Mouse", harga: 150000, stok: 0 },
51   { nama: "Keyboard", harga: 450000, stok: 12 },
52   { nama: "Monitor", harga: 3200000, stok: 0 },
53   { nama: "Headset", harga: 350000, stok: 8 },
54 ];
55
56 // 4. Filter produk yang tersedia (stok > 0)
57 const produkTersedia = produk.filter((item) => item.stok > 0);
58 console.log("Produk tersedia:", produkTersedia);
59
60 // 5. Map untuk ambil nama produk saja
61 const namaProduk = produkTersedia.map((item) => item.nama);
62 console.log("Nama produk tersedia:", namaProduk);
63
64 // 6. Reduce untuk total nilai inventaris
65 const totalInventaris = produkTersedia.reduce(
66   (total, item) => total + item.harga * item.stok,
67   0,
68 );
69 console.log("Total nilai inventaris:", totalInventaris);
70
71 // 7. forEach untuk menampilkan semua produk
```

```

Rata-rata nilai lulus: 81.17
Produk tersedia: [
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]
=== Method Chaining ===
Rata-rata nilai lulus: 81.17
Produk tersedia: [
  { nama: 'Laptop', harga: 8500000, stok: 5 },
]
=== Method Chaining ===
Rata-rata nilai lulus: 81.17
Produk tersedia: [
Rata-rata nilai lulus: 81.17
Produk tersedia: [
Produk tersedia: [
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

```

Bagian 5 — Rekursi

Rekursi adalah teknik dalam pemrograman di mana sebuah function memanggil dirinya sendiri untuk menyelesaikan suatu masalah. Rekursi biasanya digunakan untuk memecah masalah besar menjadi bagian-bagian yang lebih kecil hingga mencapai kondisi tertentu.

Setiap function rekursif terdiri dari dua komponen utama:

- **Base case (kondisi berhenti)** — yaitu kondisi saat function tidak lagi melakukan pemanggilan terhadap dirinya sendiri. Jika bagian ini tidak ada, maka rekursi akan terus berjalan tanpa henti dan dapat menyebabkan error *Maximum call stack exceeded*.
- **Recursive case (kondisi rekursif)** — yaitu bagian di mana function memanggil dirinya sendiri dengan parameter yang lebih kecil atau lebih sederhana agar mendekati kondisi berhenti.

Langkah 1 Buat file baru bernama 05-rekursi.js di dalam folder praktikum-3/

```

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
● $ touch 05-rekursi.js

```

praktikum-3	
JS 01-function-dasar.js	U
JS 02-arrow-callback.js	U
JS 03-array-dasar.js	U
JS 04-array-method.js	U
JS 05-rekursi.js	U

Langkah 2 Buka file 05-rekursi.js di VS Code, lalu ketikkan kode berikut.

```
praktikum-3 > JS 05-rekursi.js > ...
1  function faktorial(n) {
2      // Base case: berhenti saat n = 1
3      if (n <= 1) return 1;
4      // Recursive case: n x faktorial(n-1)
5      return n * faktorial(n - 1);
6  }
7
8  console.log("=== Faktorial ===");
9  console.log("0! =", faktorial(0));
10 console.log("1! =", faktorial(1));
11 console.log("5! =", faktorial(5));
12 console.log("7! =", faktorial(7));
13
14 function fibonacci(n) {
15     // Base case
16     if (n === 0) return 0;
17     if (n === 1) return 1;
18
19     // Recursive case
20     return fibonacci(n - 1) + fibonacci(n - 2);
21 }
22 console.log("\n=== Fibonacci ===");
23 for (let i = 0; i <= 8; i++) {
24     process.stdout.write(fibonacci(i) + " ");
25 }
```

Langkah 3 Simpan file, lalu jalankan dari terminal.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
● $ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
○ Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
```

Buat folder dan file tugas.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
● $ mkdir tugas

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
● $ touch tugas/tugas-1.js tugas/tugas-2.js
```

```
▼ praktikum-3
  ▼ tugas
    JS tugas-1.js
    JS tugas-2.js
```

Tugas 1

```
praktikum-3 > tugas > JS tugas-1.js > ...
1 // 1. Array dataMahasiswa (minimal 6 data)
2 const dataMahasiswa = [
3   { nama: "Andi", nilai: 85 },
4   { nama: "Budi", nilai: 70 },
5   { nama: "Citra", nilai: 90 },
6   { nama: "Dina", nilai: 60 },
7   { nama: "Eka", nilai: 75 },
8   { nama: "Fajar", nilai: 50 },
9 ];
10
11 // 2. Function hitungStatistik
12 function hitungStatistik(arrMahasiswa) {
13   const hasil = arrMahasiswa.reduce(
14     (acc, curr) => {
15       acc.total += 1;
16       acc.jumlahNilai += curr.nilai;
17
18       if (curr.nilai > acc.tertinggi) acc.tertinggi = curr.nilai;
19       if (curr.nilai < acc.terendah) acc.terendah = curr.nilai;
20
21       return acc;
22     },
```



```

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
• $ node tugas/tugas-1.js
=== Statistik ===
{ total: 6, tertinggi: 90, terendah: 50, rataRata: 71.66666666666667 }

=== Mahasiswa Lulus ===
Andi - 85
Budi - 70
Citra - 90
Eka - 75

=== Data + Grade ===
Andi - 85 (A)
Budi - 70 (C)
Citra - 90 (A)
Dina - 60 (D)
Eka - 75 (B)
Fajar - 50 (D)

```

Tugas 2

```

praktikum-3 > tugas > JS tugas-2.js > ...

```

```

1  // 1. Function rekursif pangkat
2  function pangkat(basis, eksponen) {
3      if (eksponen === 0) return 1;
4      return basis * pangkat(basis, eksponen - 1);
5  }
6
7  // Test
8  console.log("Pangkat:");
9  console.log(pangkat(2, 3)); // 8
10 console.log(pangkat(5, 2)); // 25
11
12 // 2. Function rekursif balikString
13 function balikString(str) {
14     if (str.length <= 1) return str;
15     return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
16 }
17
18 // Test
19 console.log("\nBalik String:");
20 console.log(balikString("halo")); // olah
21 console.log(balikString("level")); // level

```

```

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-3 (main)
• $ node tugas/tugas-2.js
Pangkat:
8
25

Balik String:
olah
level

```

Submit ke GitHub

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main|MERGING)
● $ git add .

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main|MERGING)
● $ git commit -m "modul"
[main fc443cc] modul

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
● $ git push
Enumerating objects: 23, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 4 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (18/18), 601.59 KiB | 1.50 MiB/s, done.
Total 18 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
```

praktikum-3	
tugas	
01-function-dasar.js	
02-arrow-callback.js	
03-array-dasar.js	
04-array-method.js	
05-rekursi.js	
package.json	
Laporan Praktikum Algoritma da...	
README.md	
intro.js	
package.json	

..	
tugas	Praktikum 3: Function & Array - arrow function, callback, itera... 4 days ago
01-function-dasar.js	Praktikum 3: Function & Array - arrow function, callback, itera... 4 days ago
02-arrow-callback.js	Praktikum 3: Function & Array - arrow function, callback, itera... 4 days ago
03-array-dasar.js	Praktikum 3: Function & Array - arrow function, callback, itera... 4 days ago
04-array-method.js	Praktikum 3: Function & Array - arrow function, callback, itera... 4 days ago
05-rekursi.js	Praktikum 3: Function & Array - arrow function, callback, itera... 4 days ago
package.json	Praktikum 3: Function & Array - arrow function, callback, itera... 4 days ago

KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pemahaman mengenai function dan array dalam JavaScript sangat penting dalam proses pengembangan program.

Function berperan dalam membuat kode menjadi lebih terstruktur, efisien, dan dapat digunakan kembali, sehingga mengurangi pengulangan kode. Arrow function memberikan kemudahan dalam penulisan function dengan sintaks yang lebih ringkas, sedangkan callback memungkinkan function digunakan secara fleksibel sebagai parameter dalam function lain.

Selain itu, array merupakan struktur data yang sangat berguna untuk menyimpan dan mengelola kumpulan data. Penggunaan method array seperti *forEach*, *map*, *filter*, dan *reduce* sangat membantu dalam memproses data secara efisien dan lebih mudah dipahami.

Rekursi juga menjadi salah satu konsep penting yang memungkinkan penyelesaian masalah secara bertahap dengan memecahnya menjadi bagian yang lebih kecil, meskipun penggunaannya harus diperhatikan agar tidak menyebabkan kesalahan seperti *stack overflow*. Dengan memahami seluruh konsep tersebut, mahasiswa diharapkan mampu mengembangkan program yang lebih baik, efisien, serta memiliki struktur yang jelas dan mudah dipelajari.